Department of Artificial Intelligence (AI) & Data Science

Name: Aaryan Nandkishor Raorane	**Roll no: S004**
Sap ID: 60018250070	**Topic: Data Structure (Searching)**
Branch: Artificial Intelligence & Data Science	

Experiment No. 9

Aim: To implement Binary Search and Fibonacci Search on a sorted list of book IDs in a digital library and to compare their search efficiency by counting the comparisons made by each technique.

Theory:

Part A: Binary Search

Definition:

Binary Search is a searching technique that works on a sorted array. The key is compared with the middle element, and the half that cannot contain the key is discarded. The search range is halved at every step, so the worst case time is O(log n). It needs random access to elements, which an array provides.

1. Binary Search (Find a Book ID)

Pseudocode:

```
BEGIN binarySearch(a, n, key)
    low = 0
    high = n - 1
    count = 0
    WHILE low <= high DO
        mid = (low + high) / 2
        count = count + 1
        IF a[mid] == key THEN
            RETURN mid
        ELSE IF key < a[mid] THEN
            high = mid - 1
        ELSE
            low = mid + 1
        END IF
    END WHILE
```

```
    RETURN -1
  END binarySearch
```

Part B: Fibonacci Search

Definition:

Fibonacci Search also works on a sorted array. Fibonacci numbers (0, 1, 1, 2, 3, 5, 8, 13 and so on, where each number is the sum of the previous two) are used to divide the range instead of halving it. The smallest Fibonacci number greater than or equal to n sets the size of the search window, and each comparison reduces the window by one or two Fibonacci steps. The probe position is found using only addition and subtraction, and the worst case time is O(log n).

1. Fibonacci Search (Find a Book ID)

Pseudocode:

```
  BEGIN fibonacciSearch(a, n, key)
    fib2 = 0
    fib1 = 1
    fib = fib2 + fib1
    offset = -1
    count = 0
    WHILE fib < n DO
      fib2 = fib1
      fib1 = fib
      fib = fib2 + fib1
    END WHILE
    WHILE fib > 1 DO
      i = MIN(offset + fib2, n - 1)
      count = count + 1
      IF a[i] < key THEN
        fib = fib1
        fib1 = fib2
        fib2 = fib - fib1
        offset = i
      ELSE IF a[i] > key THEN
        fib = fib2
        fib1 = fib1 - fib2
        fib2 = fib - fib1
      ELSE
```

```
            RETURN i
        END IF
    END WHILE
    IF fib1 == 1 AND offset + 1 < n THEN
        count = count + 1
        IF a[offset + 1] == key THEN
            RETURN offset + 1
        END IF
    END IF
    RETURN -1
END fibonacciSearch
```

Part C: Comparison of Search Efficiency

Basis	Binary Search	Fibonacci Search
Range reduction	Halves the range each time.	Reduces the range by Fibonacci steps.
Probe position	Division: (low + high) / 2.	Addition and subtraction only.
Worst case time	O(log n)	O(log n), about 1.44 times more steps.
Extra space	O(1)	O(1)

Code: Binary Search and Fibonacci Search

```
#include <stdio.h>
#include <conio.h>

int bcount, fcount;

int binarySearch(int a[], int n, int key)
{
    int low = 0, high = n - 1, mid;
    bcount = 0;
    while (low <= high)
    {
        mid = (low + high) / 2;
        bcount++;
        if (a[mid] == key)
            return mid;
        else if (key < a[mid])
```

```
            high = mid - 1;
        else
            low = mid + 1;
    }
    return -1;
}

int fibonacciSearch(int a[], int n, int key)
{
    int fib2 = 0, fib1 = 1, fib, offset = -1, i;
    fib = fib2 + fib1;
    fcount = 0;
    while (fib < n)
    {
        fib2 = fib1;
        fib1 = fib;
        fib = fib2 + fib1;
    }
    while (fib > 1)
    {
        i = offset + fib2;
        if (i > n - 1)
            i = n - 1;
        fcount++;
        if (a[i] < key)
        {
            fib = fib1;
            fib1 = fib2;
            fib2 = fib - fib1;
            offset = i;
        }
        else if (a[i] > key)
        {
            fib = fib2;
            fib1 = fib1 - fib2;
            fib2 = fib - fib1;
        }
        else
            return i;
    }
    if (fib1 == 1 && offset + 1 < n)
```

```
    {
        fcount++;
        if (a[offset + 1] == key)
            return offset + 1;
    }
    return -1;
}

void main()
{
    int a[50], n, i, key, ch, r1, r2;
    clrscr();
    printf("Enter number of book IDs: ");
    scanf("%d", &n);
    printf("Enter %d book IDs in sorted order:\n", n);
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);
    do
    {
        printf("\n1. Binary Search\n2. Fibonacci Search\n");
        printf("3. Compare Both\n4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &ch);
        if (ch >= 1 && ch <= 3)
        {
            printf("Enter book ID to search: ");
            scanf("%d", &key);
        }
        switch (ch)
        {
        case 1:
            r1 = binarySearch(a, n, key);
            if (r1 == -1)
                printf("Not found, comparisons = %d\n", bcount);
            else
            {
                printf("Found at position %d, ", r1 + 1);
                printf("comparisons = %d\n", bcount);
            }
            break;
        case 2:
```

```
            r2 = fibonacciSearch(a, n, key);
            if (r2 == -1)
                printf("Not found, comparisons = %d\n", fcount);
            else
            {
                printf("Found at position %d, ", r2 + 1);
                printf("comparisons = %d\n", fcount);
            }
            break;
        case 3:
            r1 = binarySearch(a, n, key);
            r2 = fibonacciSearch(a, n, key);
            printf("Binary Search    : ");
            if (r1 == -1)
                printf("Not found, comparisons = %d\n", bcount);
            else
                printf("Position %d, comparisons = %d\n", r1 + 1, bcount);
            printf("Fibonacci Search : ");
            if (r2 == -1)
                printf("Not found, comparisons = %d\n", fcount);
            else
                printf("Position %d, comparisons = %d\n", r2 + 1, fcount);
            break;
        case 4:
            printf("Exiting program\n");
            break;
        default:
            printf("Invalid choice\n");
        }
    } while (ch != 4);
    getch();
}
```

Output:

```
Enter number of book IDs: 15
Enter 15 book IDs in sorted order:
1001 1005 1012 1020 1034 1047 1058 1063 1079 1085 1092 1106 1118 1125 1140

1. Binary Search
2. Fibonacci Search
```

```
3. Compare Both
4. Exit
Enter your choice: 1
Enter book ID to search: 1034
Found at position 5, comparisons = 4

1. Binary Search
2. Fibonacci Search
3. Compare Both
4. Exit
Enter your choice: 2
Enter book ID to search: 1034
Found at position 5, comparisons = 3

1. Binary Search
2. Fibonacci Search
3. Compare Both
4. Exit
Enter your choice: 3
Enter book ID to search: 1099
Binary Search    : Not found, comparisons = 4
Fibonacci Search : Not found, comparisons = 5

1. Binary Search
2. Fibonacci Search
3. Compare Both
4. Exit
Enter your choice: 3
Enter book ID to search: 1125
Binary Search    : Position 14, comparisons = 3
Fibonacci Search : Position 14, comparisons = 4

1. Binary Search
2. Fibonacci Search
3. Compare Both
4. Exit
Enter your choice: 4
Exiting program
```

Observations:

The same 15 sorted book IDs were searched with both techniques.

Book ID	Position	Binary Search comparisons	Fibonacci Search comparisons
1001	1	4	3
1034	5	4	3
1063	8	1	1
1125	14	3	4
1140	15	4	3
1099 (absent)	Not found	4	5

The program was also run on 1,000 sorted IDs, searching every ID once. Binary Search averaged 8.99 comparisons with a maximum of 10. Fibonacci Search averaged 9.73 comparisons with a maximum of 14.

Applications:

1. Binary Search Applications:

1. Library Catalogue Lookup: Book IDs are kept in sorted order, so a student can find a book in about log n comparisons even when the catalogue holds thousands of records.

2. Database and File Indexes: Sorted index entries are searched with Binary Search to reach a record without scanning the whole file.

2. Fibonacci Search Applications:

1. Sequential and Tape Storage: Where the cost of access grows with distance, Fibonacci Search keeps the probes close together and avoids long jumps.

2. Systems with Costly Division: The probe position uses only addition and subtraction, which suits hardware where division is expensive.

Conclusion:

This experiment implemented Binary Search and Fibonacci Search on sorted book IDs and compared them by counting comparisons. Both techniques need a sorted array and both run in O(log n) time. On 1,000 IDs Binary Search made fewer

comparisons on average and in the worst case, so it is the better choice for the library system.